Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2026
Introduction to Software Engineering

CourseCompass: Academic Course & Schedule Planner

Software Requirements Specification (SRS) - Phase 2

Team Members:
Habiba Emam Fathi 202510585
Bassant Salem 202507533
Rahma Mohsen 202507049
Representative Contact info:
s-bassant.meselhy@zewailcity.edu.eg

Table of Contents

1. Introduction *3*

1.1 Purpose 3

1.2 Scope 3

1.3 Definitions, Acronyms, and Abbreviations 3

1.4 References 3

1.5 Overview 3

2. Overall Description *4*

2.1 Product Perspective 4

2.2 Product Functions 4

2.3 User Classes and Characteristics 4

2.4 Software Stakeholders 4

2.5 Operating Environment 4

2.6 Design and Implementation Constraints 4

2.7 User Documentation 5

2.8 Assumptions and Dependencies 5

3. Specific Requirements *5*

3.1 Functional Requirements 5

3.2 Use Case Model 7

3.1.1 Use Case Diagram 7

3.1.2 Use Case Descriptions 9

3.3 Domain Model 20

3.3.1 Conceptual Class Diagram 20

3.3.2 Class Descriptions 21

3.4 Non-Functional Requirements 23

3.5 External Interface Requirements 24

3.5.1 User Interface 24

3.5.2 Hardware Interface 25

3.5.3 Software Interface 25

3.5.4 Communication Interface 25

4. Appendices *25*

4.1 Appendix A: Data Dictionary 25

4.2 Appendix B: Glossary 25

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) of **CourseCompass**, a web-based Academic Course & Schedule Planner. It states what the system must do before design and implementation start. It is written for the development team, who will use it as the baseline for the design, coding and testing phases, and for the TAs and the instructor, who will use it to evaluate and trace later deliverables.

1.2 Scope

The Academic Course & Schedule Planner is a web-based application designed to streamline the academic registration process. The system allows students to build semester conflict-free timetables, track prerequisite requirements, and calculate GPAs. Simultaneously, it provides administrators with the tools to manage course offerings, class capacities, and prerequisites. The ultimate goal is to eliminate manual scheduling errors and provide a centralized, efficient platform for university course management

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
SRS / UML / MVC	Software Requirements Specification / Unified Modeling Language / Model-View-Controller
GPA	Grade Point Average (4.0 scale)
Section	A scheduled offering of a course in one semester (time slots, room, instructor, capacity)
Prerequisite	A course (with a minimum grade) that must be completed before taking another course

1.4 References

[1] CSAI 203 Course Project description, Fall 2026, Dr. Mohamed Sami Rakha.
[2] https://ieeexplore.ieee.org/document/720574.
[3] https://developer.mozilla.org/en-US/
[4] https://lucid.co/lcampaign=ixed_search
[5] https://app.diagrams.net/

1.5 Overview

The remainder of this document is organized as follows: Section 2 provides a comprehensive description of the system,including user characteristics, operating environment, and constraints. Section 3 details the specific functional and non-functional requirements, use cases, and conceptual models necessary to build the software.

2. Overall Description

2.1 Product Perspective

The Academic Course & Schedule Planner is a new, standalone web-based application. It is designed from scratch to operate independently of legacy systems. The software architecture will strictly adhere to the Model-View-Controller (MVC) pattern, separating the data logic (Model), user interface (View), and input control (Controller)

2.2 Product Functions

- **Account and access:** registration, login, password reset , student academic record.
- **Catalog:** search, filter and view courses, sections, prerequisites and seat availability.
- **Planning:** timetable builder, conflict detection, prerequisite and credit-limit checks.
- **Registration:** register, drop, degree progress tracking, notifications.
- **GPA:** semester/cumulative GPA calculation .
- **Administration:** manage courses, sections, prerequisites, users, semesters, registration periods.

2.3 User Classes and Characteristics

User class	Characteristics	Privileges
Student	Undergraduate, basic web skills, uses the system mainly before and during registration weeks.	use the system to browse courses, plan their semesters, and track their academic progress.
Administrator	University staff members or academic advisors	manage the course database, adjust capacities, and oversee the registration process

2.4 Software Stakeholders

Stakeholder	Interest in the software
Students	Schedule building, GPA calculation
Registrar / administrators	System Managers‹Course offerings, capacity management
Academic advisors	Ability to check that student plans respect prerequisites and degree requirements.
University IT	Secure, maintainable system that is simple to deploy.
Course instructor and TAs	Course EducatorsFinal student rosters, class sizes

2.5 Operating Environment

The system is a web application that will operate on standard web browsers (e.g., Google Chrome, Mozilla Firefox,Safari, Microsoft Edge). The backend will be hosted on a standard web server with a connected relational database to store user and course information. It requires an active internet connection.

2.6 Design and Implementation Constraints

The MVC pattern is mandatory and must be visible in the design and the code. Only the frameworks approved by the course may be used (backend and frontend). Source code, docs, tests and deployment files follow the required GitHub structure (/docs, /src, /tests,

/deployment). Auto-code-generation tools are not allowed. All traffic uses HTTPS, passwords are stored hashed, and student data follows the university's privacy rules.

2.7 User Documentation

An in-app FAQ page, short tooltips on complex screens (timetable grid, prerequisite map), a one-page student guide and a short administrator guide (PDF), plus the README for deployment.

2.8 Assumptions and Dependencies

Assumptions: Users have internet-connected devices with modern browsers, possess valid university accounts, and administrators have correctly configured all course data and prerequisites in advance.

Dependencies: The system relies on a secure API connection to the university's primary database to fetch student transcripts, a third-party email service for waitlist alerts, and a stable cloud server to handle concurrent traffic during registration.

3. Specific Requirements

3.1 Functional Requirements

FR-01-Account Activation
The system shall allow students to activate their pre-created institutional accounts using their university ID and official email. Scenario: A student enters their university ID and email, verifies through an activation link, sets a secure password,and gains access to the system.

FR-02 - Authentication and Session
Users log in with email (or ID) and password and are redirected to a dashboard that matches their role. Sessions expire after 30 minutes of inactivity, and logout ends the session at once. After 5 consecutive failed attempts the account is locked for 15 minutes. *Scenarios:* wrong password gives a generic message (never revealing which field failed); a deactivated account is told to contact the administrator.

FR-03 - Password Reset

Password Recovery: Users shall be able to reset forgotten passwords via an email link. *Scenario:* A student clicks "Forgot Password", receives a reset link, and establishes a new secure password.

FR-04 Course Catalog Browsing
Students shall be able to view a complete list of offered courses for the upcoming semester.*Scenario:* A student navigates to the catalog and views course titles, codes, credit hours, and available sections

FR-05 - Student Profile and Academic Record (UC-01, UC-13). A student edits personal data and maintains a record of completed courses with grades and semester. This record feeds prerequisite checks, degree progress and GPA. *Scenarios:* adding "CSAI101

A-" immediately unlocks courses that require it; adding the same course twice is treated as a repeat (latest grade counts).

FR-06 - Course Search & Filtering: The system shall allow users to search for courses by name, code, or department. *Scenario:*A student types "CSAI" to filter and view only computer science and AI courses.

FR-07 - Course Details View
A course page shows description, credit hours, prerequisite and corequisite expression, and all sections of the current semester with days, times, room, instructor and remaining seats. A button adds a section to the active draft. *Scenarios:* an ineligible student sees the missing prerequisites highlighted; an archived course shows a notice instead of sections.

FR-08 -Remove Course from Draft

Students shall be able to remove a course section from their draft timetable before final registration submission. Scenario:A student decides against a morning class draft and clicks "Remove", freeing up that time slot in their active draft.

FR-09 - Time-Conflict Detection
When a section is added, its time slots are compared with every section in the draft on each weekday. Overlaps are blocked and the clashing sections are highlighted; the system proposes alternative non-conflicting sections of the same course

FR-10 Capacity Enforcement
The system shall prevent registration if a course section reaches its maximum student limit. *Scenario:* A course has a capacity of 50. The 51st student attempting to register is blocked and offered a waitlist option

FR-11 - Prerequisite Tracking and Degree Progress (UC-05, UC-08). A progress map of the student's program marks each course as completed, in progress, eligible or locked, and shows earned versus required credits. *Scenario:* clicking a locked course highlights the chain of prerequisites that must be finished first.

FR-12 - Credit-Hour Limit Validation
A draft must stay between the minimum and maximum credit hours set by the administrator (default 12 to 18). Registration is blocked outside this range and the message states how many credits to add or remove.

FR-13- Course Registration (UC-09)
Submitting a draft inside the registration window checks, for every section, prerequisites, conflicts, credit limits and seat availability, then creates enrollments and decreases seats in one atomic step. A summary lists accepted and rejected sections with reasons. *Scenarios:* if the last seat is taken by another student first, the section is rejected as full

FR-14 - Drop Course

Before the drop deadline a student can drop an enrolled course. The system shows the effect on total credits, requires confirmation if the total falls below the minimum, frees the seat and triggers the waitlist. After the deadline the request is rejected.

FR-15 - Timetable Export and Print
A draft or registered schedule can be exported as a PDF or an iCalendar (.ics) file containing course codes, times, rooms and instructors, or printed with a print-friendly layout.

FR-16- GPA Calculator
Semester and cumulative GPA are computed on a 4.0 scale as the sum of grade points times credits divided by total graded credits. Withdrawn and incomplete courses are excluded and repeated courses use the latest grade. *Scenario:* a student enters expected grades for current courses to see the resulting cumulative GPA.

FR-17 - What-If GPA Projection
A student enters a target cumulative GPA and the number of remaining credits; the system returns the average semester GPA needed, or states that the target is unreachable (needed value above 4.0).

FR-18- Admin Course and Section Management
Administrators create, edit and archive courses and sections (code, title, credits, department, semester, instructor, room, time slots, capacity). The system rejects room or instructor double-booking, refuses a capacity below current enrollment, and archives instead of deleting courses that have enrollments.

FR-19 - User Management
Administrators search users, activate or deactivate accounts, change roles and trigger password resets. The last active administrator account cannot be deactivated.

FR-20 - Notifications
The system sends in-app and email notifications for password reset, waitlist seat offers, registration confirmation, and schedule changes made by administrators. *Scenario:* If an administrator changes the room location for a specific course section, all enrolled students receive an immediate email notification, and their personalized schedules are automatically updated with the new venue.

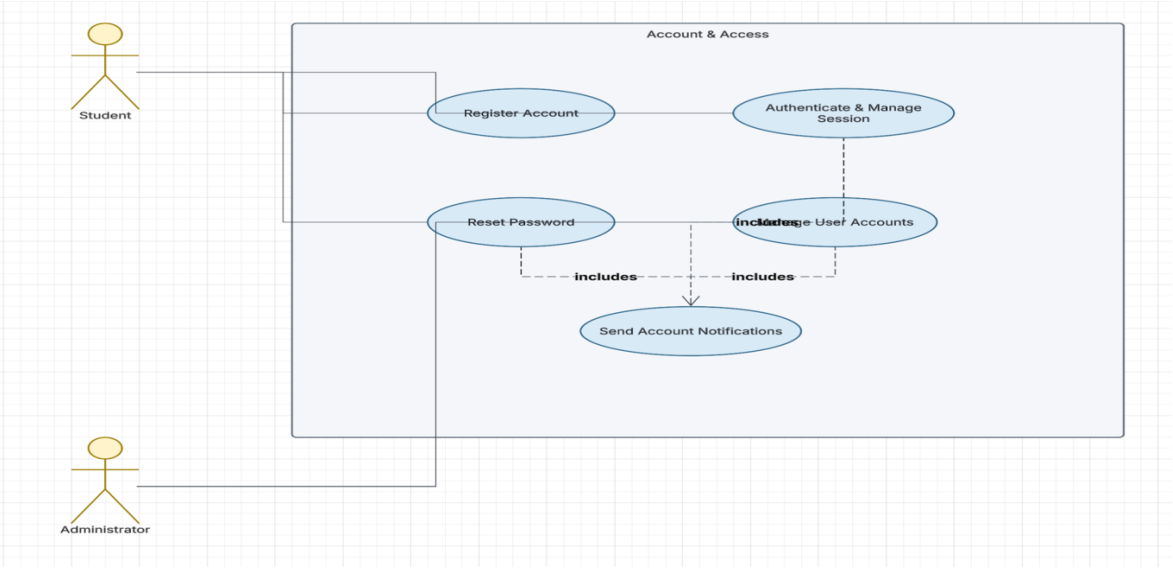
3.2 Use Case Model

3.1.1 Use Case Diagram

Overall system goal: provide a secure, consistent course-registration workflow that helps students plan their studies and register successfully while allowing administrators to maintain academic data and operational rules..

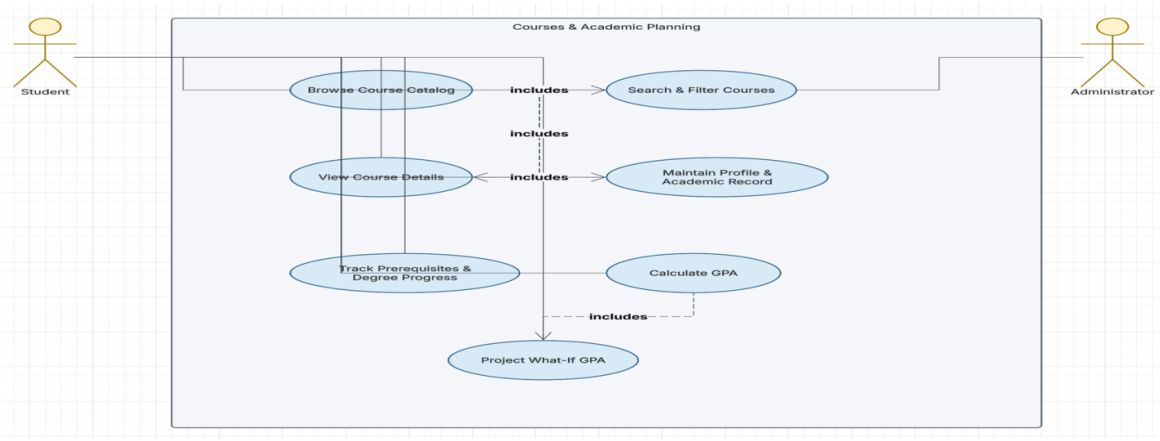
1. **Account and Access**

- Students create accounts with university email and ID.
- Users log in, receive role-based dashboards, and can log out.
- Sessions expire after 30 minutes of inactivity.
- Accounts lock after five failed login attempts for 15 minutes.
- Users can reset forgotten passwords by email.
- Administrators manage users, roles, activation status, and password rese



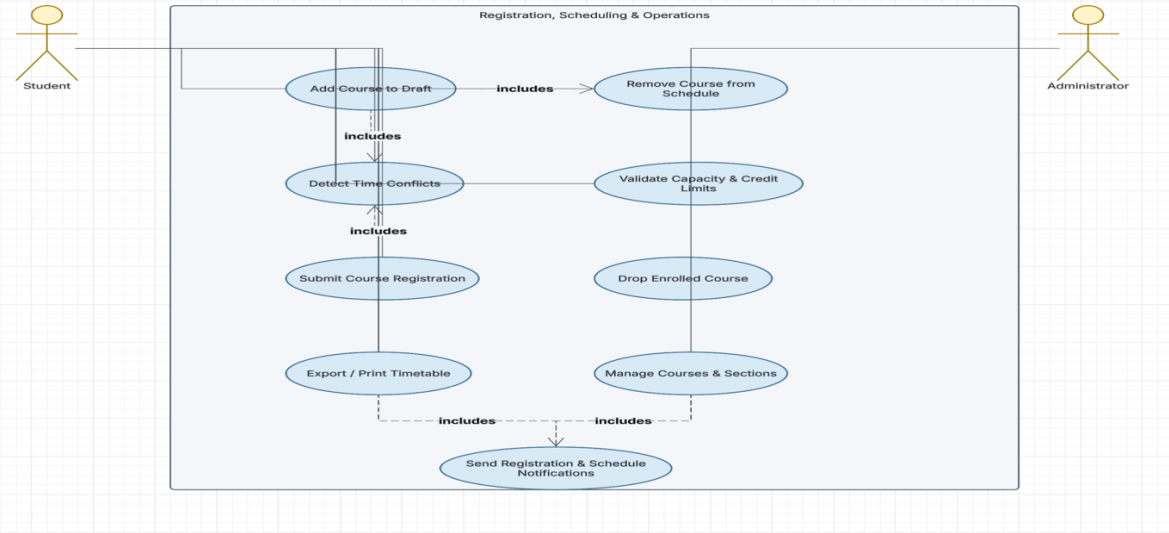
2.Courses and Academic Planning

- Students browse, search, and filter the course catalog.
- Course details include credits, prerequisites, corequisites, sections, instructors, rooms, times, and remaining seats.
- Students maintain their profiles and academic records.
- The system tracks prerequisites, degree progress, completed courses, repeats, and GPA.
- Students can calculate GPA and perform what-if GPA projections.



3.Registration, Scheduling, and Operations

- Students add or remove courses from a draft schedule.
- The system detects time conflicts and suggests alternatives.
- Registration validates prerequisites, seat capacity, credit limits, and registration dates.
- Students can drop courses before the deadline.
- Schedules can be exported as PDF or iCalendar files and printed.
- Administrators manage courses and sections while preventing room/instructor conflicts.
- Notifications are sent for password resets, registration results, waitlist offers, and schedule changes.



3.1.2 Use Case Descriptions

Use Case ID	UC-01	Related FR	FR-01
Actor(s)	Student		
Use Case Name	Account Activation		
Preconditions	A pre-created institutional account exists; the student has a valid university ID and official email.		
Main flow	1. The student opens the registration page. 2. The system displays the registration form. 3. The student enters the required information.		

	4. The system validates the information. 5. The system sends an email verification link. 6. The student opens the link. 7. The system verifies the email and activates the account. 8. The system confirms successful registration.
Alternative / exception flows	A1. Invalid email or ID: the system displays a validation error. A2. Duplicate email or ID: the system rejects registration. A3. Invalid or expired verification link: the system allows a new link to be requested.
Postconditions	A verified student account is created and available for login.

Use Case ID	UC-02	Related FR	FR-02
Actor(s)	Student, Administrator		
Use Case Name	Authentication and Session Management		
Preconditions	The user has an existing account.		
Main flow	1. The user opens the login page. 2. The user enters an email or university ID and password. 3. The system validates credentials and account status. 4. The system creates an authenticated session. 5. The system redirects the user to the appropriate role-based dashboard. 6. The system monitors inactivity. 7. On logout, the system terminates the session immediately.		
Alternative / exception flows	A1. Incorrect credentials: display a generic error message. A2. Five consecutive failed attempts: lock the account for 15 minutes. A3. Deactivated account: instruct the user to contact the administrator. A4. Thirty minutes of inactivity: expire the session and require reauthentication.		
Postconditions	The user is authenticated with an active session or denied access.		

Use Case ID	UC-03	Related FR	FR-03
Actor(s)	Student, Administrator		

Use Case Name	Password Reset
Preconditions	The user has an existing account and access to the registered email address.
Main flow	1. The user requests a password reset. 2. The system requests the registered email address. 3. The user submits the email address. 4. The system sends a reset link when applicable. 5. The user opens the link. 6. The system validates the link. 7. The user enters and confirms a new password. 8. The system validates and saves the password
Alternative / exception flows	A1. Invalid or expired link: request a new link. A2. Weak password: display the password requirements. A3. Unknown email: avoid unnecessarily revealing whether an account exists.
Postconditions	The user's password is updated if the reset succeeds

Use Case ID	UC-04	Related FR	FR-04
Actor(s)	Student		
Use Case Name	Browse Course Catalog		
Preconditions	The course catalog is available		
Main flow	1. The student opens the catalog. 2. The system retrieves courses offered for the upcoming semester. 3. The system displays course titles, codes, credit hours, and available sections. 4. The student browses the catalog. 5. The student may select a course to view its details.		
Alternative / exception flows	A1. No courses are available: display an appropriate message. A2. Catalog retrieval fails: display an error and allow retry.		
Postconditions	The available course catalog is displayed.		

Use Case ID	UC-05	Related FR	FR-05
Actor(s)	Student		
Use Case Name	Manage Student Profile and Academic Record		
Preconditions	The student is authenticated		

Main flow	1. The student opens the profile. 2. The system displays personal information and academic records. 3. The student edits permitted personal information or adds a completed cou rse, grade, and semester. 4. The system validates the data. 5. The system checks for duplicate course records. 6. The system saves valid changes. 7. The system updates prerequisite checks, degree progress, and GPA calcul ations. 8. The system confirms the changes.
Alternative / exception flows	A1. Missing or invalid data: display validation errors. A2. Repeated course: treat it as a repeat and apply the latest-grade rule. A3. Unauthorized modification: reject the operation.
Postconditi ons	The academic record and permitted profile data are updated

Use Case ID	UC-06	Related FR	FR-06
Actor(s)	Student		
Use Case Name	Search and Filter Courses		
Preconditions	The course catalog is accessible.		
Main flow	1. The student opens the catalog. 2. The student enters a course name, code, or department. 3. The system searches the catalog. 4. The system displays matching courses. 5. The student selects a result or changes the search.		
Alternative / exception flows	A1. No matches: display “No courses found.” A2. Empty search: display the full catalog or request a search term.		
Postconditions	Matching courses are displayed		

Use Case ID	UC-07	Related FR	FR-07
Actor(s)	Student		
Use Case Name	View Course Details and Add Section to Draft		

Preconditions	The student can access the catalog, and the course exists.
Main flow	1. The student selects a course. 2. The system displays the description, credits, prerequisites, and corequisites. 3. The system displays section days, times, rooms, instructors, and remaining seats. 4. The system checks prerequisite eligibility. 5. The student selects a section and clicks “Add to Draft.” 6. The system validates the addition. 7. The system adds the valid section and updates the draft timetable.
Alternative / exception flows	A1. Missing prerequisites: highlight the requirements and prevent an ineligible addition. A2. Archived course: display an archive notice instead of current sections. A3. Full or conflicting section: explain the problem and offer alternatives when available.
Postconditions	Course details are displayed, and an eligible section may be added to the draft

Use Case ID	UC-08	Related FR	FR-08
Actor(s)	Student		
Use Case Name	Remove Course from Draft		
Preconditions	The draft schedule contains the selected section		
Main flow	1. The student opens the draft schedule. 2. The system displays the selected sections and credit total. 3. The student selects a section to remove. 4. The system requests confirmation if required. 5. The student confirms. 6. The system removes the section. 7. The system recalculates credits and updates the timetable. .		
Alternative / exception flows	A1. Student cancels: the draft remains unchanged. A2. Section not found: display an appropriate message. A3. Already registered enrollment: use the separate Drop Registered Course process.		
Postconditions	The section is removed from the draft, and totals are recalculated.		

Use Case ID	UC-09	Related FR	FR-09
Actor(s)	Student		
Use Case Name	Detect Schedule Time Conflicts		
Preconditions	The student has a draft schedule or is adding a section.		
Main flow	1. The student selects a section. 2. The system retrieves its meeting days and times. 3. The system compares the section against all sections in the draft for each weekday. 4. The system checks for overlapping times. 5. If no conflict exists, the system adds the section.		

	6. The system updates the timetable.
Alternative / exception flows	A1. Overlap detected: block the addition and highlight conflicting sections. A2. Alternative sections available: propose non-conflicting sections of the same course. A3. No alternatives: inform the student.
Postconditions	The addition is accepted only if it does not create a time conflict.

Use Case ID	UC-10	Related FR	FR-10
Actor(s)	Student		
Use Case Name	Enforce Section Capacity		
Preconditions	The selected section has a defined maximum capacity.		
Main flow	1. The student submits a registration request. 2. The system checks current enrollment and maximum capacity. 3. The system verifies seat availability at the point of enrollment. 4. If a seat is available, the system creates the enrollment and reserves the seat within the registration transaction. 5. The system updates the remaining-seat count.		
Alternative / exception flows	A1. Section full: reject enrollment and explain the reason. A2. Waitlist supported: offer the student the waitlist option. A3. Last seat taken by another student: reject the enrollment as full.		
Postconditions	Enrollment never exceeds the section's maximum capacity		

Use Case ID	UC-11	Related FR	FR-11
Actor(s)	Student		
Use Case Name	Track Prerequisites and Degree Progress		
Preconditions	The student is authenticated, and program requirements are configured.		
Main flow	1. The student opens degree progress. 2. The system retrieves academic records and program requirements. 3. The system evaluates course completion and eligibility. 4. The system labels courses as completed, in progress, eligible, or locked. 5. The system calculates earned and required credits. 6. The student selects a locked course.		

	7. The system highlights the prerequisite chain.
Alternative / exception flows	A1. Incomplete academic records: identify missing information. A2. Program requirements unavailable: display an appropriate
Postconditions	The student's course eligibility and degree progress are displayed.

Use Case ID	UC-12	Related FR	FR-12
Actor(s)	Student		
Use Case Name	Validate Credit-Hour Limits		
Preconditions	A draft exists, and credit-hour limits are configured		
Main flow	1. The student modifies or submits the draft. 2. The system calculates total credit hours. 3. The system retrieves the minimum and maximum limits. 4. The system compares the total against the permitted range. 5. If valid, the system marks the validation as successful.		
Alternative / exception flows	A1. Below minimum: block final registration and state how many credits must be added. A2. Above maximum: block final registration and state how many credits must be removed. A3. No custom limits: use the default range of 12–18 credits.		
Postconditions	The draft's credit-hour compliance is determined.		

Use Case ID	UC-13	Related FR	FR-13
Actor(s)	Student		
Use Case Name	Submit Course Registration		
Preconditions	The student is authenticated, has a draft, and the registration window is open.		
Main flow	1. The student reviews the draft. 2. The student submits the registration. 3. The system checks the registration window. 4. The system validates credit-hour limits.		

	5. The system checks prerequisites and corequisites. 6. The system checks time conflicts and seat availability. 7. The system processes enrollment changes atomically. 8. The system creates accepted enrollments and updates seat counts. 9. The system displays accepted and rejected sections with reasons. 10. The system sends a registration confirmation notification.
Alternative / exception flows	A1. Registration closed: reject submission. A2. Section full: reject the affected enrollment and offer waitlisting if supp orted. A3. Invalid prerequisites, corequisites, conflicts, or credit limits: report the reason. A4. Transaction failure: roll back uncommitted changes. A5. Mixed valid and invalid sections: report each result and process accepted sections according to the defined transaction rules
Postconditio ns	Valid enrollments are committed, seats are updated, and a registration summary is displayed

Use Case ID	UC-14	Related FR	FR-14
Actor(s)	Student		
Use Case Name	Drop Registered Course		
Preconditions	The student is authenticated and enrolled in the selected course.		
Main flow	1. The student opens the registered schedule. 2. The student selects an enrolled course. 3. The system checks the drop deadline. 4. The system calculates remaining credits. 5. If credits fall below the minimum, the system warns the student. 6. The student confirms the drop. 7. The system cancels enrollment and updates the schedule and credits. 8. The system releases the seat and triggers the waitlist process if applicable. 9. The system confirms the drop..		

Alternative / exception flows	A1. Deadline passed: reject the request. A2. Student cancels: retain the enrollment. A3. Student does not confirm the below-minimum credit warning: retain the enrollment. A4. Student is not enrolled: display an error..
Postconditions	The enrollment is cancelled, credits are recalculated, and the seat is released.

Use Case ID	UC-15	Related FR	FR-15
Actor(s)	Student		
Use Case Name	Export and Print Timetable		
Preconditions	The student has a draft or registered schedule.		
Main flow	1. The student opens the timetable. 2. The system displays the schedule. 3. The student selects PDF, iCalendar (.ics), or print. 4. The system prepares course codes, times, rooms, and instructors. 5. The system generates the requested file or print-friendly layout. 6. The student saves the file or proceeds to print.		
Alternative / exception flows	A1. Empty schedule: inform the student. A2. File-generation failure: display an error and allow retry. A3. Unsupported format: request a supported format. .		
Postconditions	The timetable is exported or prepared for printing		

Use Case ID	UC-16	Related FR	FR-16
Actor(s)	Student		
Use Case Name	Calculate GPA		
Preconditions	Academic records contain valid grades and credit hours.		
Main flow	1. The student opens the GPA calculator. 2. The system retrieves course grades and credits. 3. The system excludes withdrawn and incomplete courses. 4. The system applies the latest-grade rule to repeated courses. 5. The system calculates semester and cumulative GPA. 6. The student may enter expected grades. 7. The system displays the projected GPA.		

Alternative / exception flows	A1. Missing grades or credits: identify incomplete records. A2. Invalid expected grade: request a valid grade. A3. No graded credits: indicate that GPA cannot yet be calculated.
Postconditions	The calculated or projected GPA is displayed.

Use Case ID	UC-17	Related FR	FR-17
Actor(s)	Student		
Use Case Name	Project What-If GPA		
Preconditions	The student has a valid current cumulative GPA and completed-credit total.		
Main flow	1. The student opens the tool. 2. The system retrieves the current GPA and graded credits. 3. The student enters a target GPA and remaining graded credits. 4. The system validates the inputs. 5. The system calculates the required average GPA. 6. The system displays the result.		
Alternative / exception flows	A1. Invalid target or credits: request valid inputs. A2. Required average exceeds 4.0: state that the target is unreachable under the assumed credit totals. A3. Required average is within 0.0–4.0: display the required average.		
Postconditions	The required average GPA or an unreachable-target message is displayed.		
Include / Extend	None		

Use Case ID	UC-18	Related FR	FR-18
Actor(s)	Administrator		
Use Case Name	Manage Courses and Sections.		
Preconditions	The administrator is authenticated and authorized		
Main flow	1. The administrator opens course management. 2. The system displays existing courses and sections. 3. The administrator selects create, edit, or archive. 4. The administrator enters course and section details.		

	5. The system validates the information. 6. The system checks room and instructor scheduling conflicts. 7. The system checks that capacity is not below current enrollment. 8. The system saves valid changes. 9. The system confirms the operation
Alternative / exception flows	A1. Room or instructor double-booking: reject the conflicting change. A2. Capacity below current enrollment: reject the change. A3. Course has enrollments: archive instead of permanently deleting it. A4. Invalid or missing data: display validation errors. A5. Schedule changes: initiate notifications to affected students.
Postconditions	Valid course or section changes are saved, or the operation is rejected

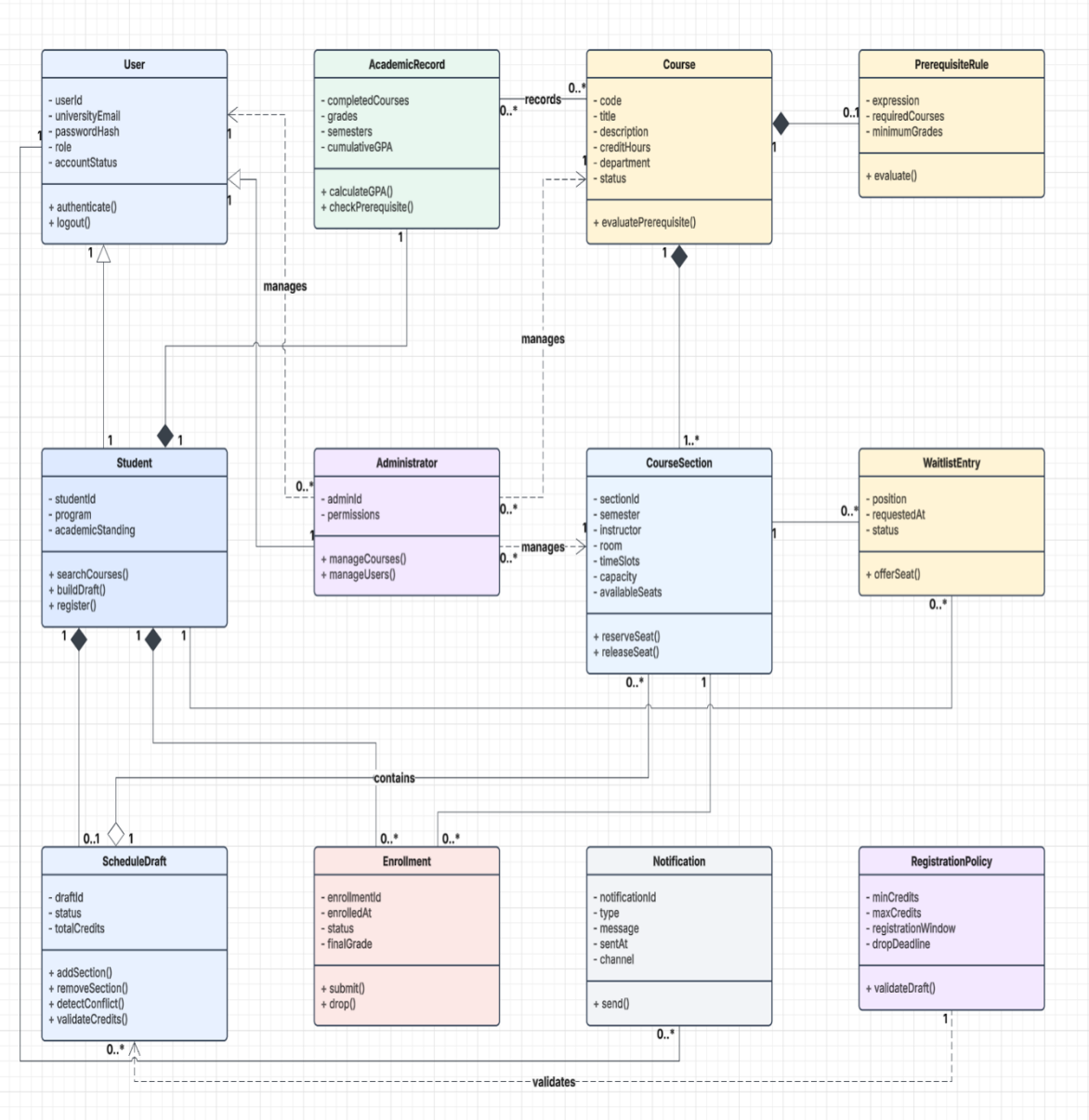
Use Case ID	UC-19	Related FR	FR-19
Actor(s)	Administrator		
Use Case Name	Manage Users		
Preconditions	The administrator is authenticated and authorized.		
Main flow	1. The administrator opens user management. 2. The administrator searches for a user. 3. The system displays matching accounts. 4. The administrator selects an account. 5. The administrator activates or deactivates the account, changes its role, or triggers a password reset. 6. The system validates the requested operation. 7. The system saves valid changes. 8. The system confirms the operation.		
Alternative / exception flows	A1. No matching user: display “No users found.” A2. Attempt to deactivate the last active administrator: reject the operation. A3. Unauthorized or invalid change: reject the operation. A4. Password reset triggered: initiate the password-reset process.		
Postconditions	The valid user-management changes are saved.		

Use Case ID	UC-20	Related FR	FR-20

Actor(s)	System
Supporting Actors	Administrator, Student, Email Service
Use Case Name	Manage Notifications
Preconditions	A notification-triggering event occurs, and recipient information is available
Main flow	1. The system detects the event. 2. The system identifies the recipients. 3. The system prepares the notification content and delivery channels. 4. The system generates the notification. 5. The system sends the applicable in-app notification and/or email. 6. For schedule changes, the system updates section details and affected personalized timetables. 7. The system emails all enrolled students about the change. 8. The system records the notification outcome.
Alternative / exception flows	A1. Email service unavailable: queue or retry the notification and record the failure. A2. No enrolled students: update the section without sending enrollment-based notifications. A3. Invalid email: record the failure and preserve available in-app notification delivery. A4. Delivery failure: do not report the email as successfully delivered.
Postconditions	Notifications are generated and their delivery status is recorded.

3.3Domain Model

3.3.1Conceptual Class Diagram



3.3.2Class Descriptions

Purpose	Class	Attributes	Associations
Central entity for account data, authentication, and session management.	User	userId, universityEmail passwordHash role, accountStatus	Superclass of Student and Administrator. Receives 0..* Notification.
Represents a learner who searches courses, builds drafts, and registers.	Student	studentId Program Academic Standing	Subclass of User. Manages 1 AcademicRecord. Has 0..1 ScheduleDraft. Has 0..* Enrollment and 0..* WaitlistEntry

Tracks completed courses, grades, semesters, and calculates cumulative GPA.	Administrator	adminId, permissions	Subclass of User. Manages 0..* User, 0..* Course, and 0..* CourseSection
Tracks completed courses, grades, semesters, and calculates cumulative GPA.	Academic Record	completedCourses, grades, semesters, cumulativeGPA	Managed by 1 Student. Records 0..* Course.
Represents a catalog entry for a subject, independent of a specific semester	Course	code, title, creditHours, department description status	Managed by Administrator. Has 1..* CourseSection. Has 0..1 PrerequisiteRule. Recorded in 0..* AcademicRecord
Defines the rules and required prior courses needed to enroll in a specific course.	PrerequisiteRule	expression, requiredCourses, minimumGrades	Belongs to 1 Course
Represents a schedulable offering of a course with specific times and capacity.	CourseSection	sectionId, semester, instructor, room, timeSlots, capacity, availableSeats	Belongs to 1 Course. Managed by Administrator. Contained in 0..* ScheduleDraft. Has 0..* Enrollment and 0..* WaitlistEntry
Belongs to 1 Course. Managed by Administrator. Contained in 0..* ScheduleDraft.	ScheduleDraft	draftId, status, totalCredits	Belongs to 1 Student (as 0..1 association). Contains 0..* CourseSection. Validated by 1 RegistrationPolicy

Has 0..* Enrollment and 0..* WaitlistEntry			
Represents a finalized, official registration of a student in a specific section.	Enrollment	enrollmentId, enrolledAt, status, finalGrade	Associates 1 Student with 1 CourseSection.
Represents a student's queue position for a course section that is currently full.	WaitlistEntry	position, requestedAt, status	Associates 1 Student with 1 CourseSection
Represents system alerts and messages sent to users regarding schedules or accounts	Notification	notificationId, type, message, sentAt, channel	Sent to 1 User
Defines global constraints such as credit limits and registration timelines.	Registration Policy	minCredits maxCredits registrationWindow, dropDeadline	Validates 0..* ScheduleDraft.

3.4Non-Functional Requirements

ID	Requirement	How it will be tested	Success Criteria
NFR-01 Performance	Search, timetable updates and conflict checks respond quickly under load.	Load test (JMeter or Locust) with 200 concurrent users on a database of 5,000 sections.	System responds within 2 seconds for 95% of all requests.
NFR-02 Security	Passwords hashed, HTTPS only, role-based access, protection from SQL	Inspect the database; send admin requests with a student session; run OWASP ZAP baseline scan.	100% of passwords stored as hashes. 0% unauthorized

			access during role testing.
NFR-03 Usability	Responsive, intuitive interface on phones and desktops.	task test on multiple screen sizes.	Users can complete course registration within 3 clicks from the dashboard without horizontal scrolling on mobile.
NFR-04 Reliability / Integrity	Seats never over-allocated and failed operations are rolled back.	50 parallel registration requests for the last seat..	Only 1 enrollment is created; exactly 49 requests are properly rejected or waitlisted.
NFR-05 Correctness	GPA, credit and conflict rules give exact results.	Automated unit tests against expected values.	100% pass rate for all GPA and credit calculation unit tests.
NFR-06 Compatibility	Works on current mainstream browsers.	Manual flows on Chrome, Firefox, Edge, and Safari.	Core workflows function without UI breakage or console errors across the latest versions of the 4 browsers.

3.5External Interface Requirements

3.5.1 User Interface

The system will feature a responsive, web-based user interface accessible via modern browsers on both desktop and mobile devices. The UI will adopt a clean, intuitive layout utilizing a shared navigation bar across all pages. Key interfaces include:

- **Login Screen:** A secure authentication page for user access.
- **Role-Based Dashboards:** Distinct landing pages for students and administrators, presenting relevant summaries and quick actions.
- **Course Catalog & Timetable Builder:** An interactive grid and search interface allowing students to filter courses,view details, and dynamically build their draft schedules.
- **Admin Console:** Data management tables and forms for administrators to manage user roles, course offerings, and system configurations.

3.5.2Hardware Interface

None. Uses the user's device and browser's print function.

3.5.3Software Interface

Requires a relational database, SMTP email service, PDF/.ics libraries, and the host OS

3.5.4Communication Interface

The system relies on the HTTPS protocol (using TLS 1.2+ encryption standards) to secure all communications between the user's browser and the server, ensuring the protection of academic and personal data. Data exchange between the frontend and backend is handled using JSON format to guarantee fast and lightweight transmission. Additionally, the system utilizes the SMTP protocol over TLS to securely send automated notifications and emails to users.

3.2

4. Appendices

4.1 Appendix A: Data Dictionary

Table	Description
users	Account data of all users: id, name, email, password hash, role, status.
students / programs	Student-specific data and degree programs with required credits.
courses / course_prerequisites	Catalog courses and prerequisite rules (course, required course, group, minimum grade).
sections / time_slots	Offered sections (semester, instructor, room, capacity, available seats); meeting times (weekday, start, end) of each section.
registration_policy	Minimum and maximum credits, registration window and drop deadline.
enrollments / waitlist_entries	Registrations with grades and waitlist queue entries.
completed_courses	Student academic record entries (course, grade, semester).
notifications / audit_log	Messages sent to users and administrative changes history.

4.2 Appendix B: Glossary

- **Corequisite:** a course that must be taken together with another.
- **Credit hour:** unit of course workload counted toward the degree.
- **Drop deadline:** last date to withdraw without penalty.
- **Draft (ScheduleDraft):** a student's temporary timetable that becomes official only after successful registration.
- **Eligibility:** state of meeting all prerequisites of a course.
- **Enrollment:** an official registration of a student in a section.
- **Registration window:** the period during which students may submit registrations.
- **Seat:** one unit of section capacity.
- **Time slot:** a weekday with start and end time.
- **Waitlist:** queue of students waiting for a seat in a full section.